Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 3

Tələbənin adı və soyadı: Muhəmməd Abdullayev

Qrup: KOM25A

Müəllim: Şamil Hümbətov

Tarix: 29 oktyabr 2026

2026

Məqsəd

Bu laboratoriya işinin məqsədi növbə (queue) strukturunun FIFO prinsipini aptek ssenarisi, prioritet növbənin isə avtobus dayanacağı ssenarisi üzərində Python-da sıfırdan implementasiya etməkdir. İkinci məqsəd əməliyyat vaxtlarını ölçməklə O(1) nəzəri mürəkkəbliyini eksperimental sürətdə yoxlamaqdır. Üçüncü məqsəd isə sosial prioritetin (65 yaşdan yuxarı sərnişinlər) iki FIFO növbəsi vasitəsilə necə modelləşdirildiyini göstərməkdir.

Giriş

Bu işdə Queue (FIFO) və prioritet növbə strukturları Python-da sıfırdan yazılıb və 100, 1000, 10000 element üzərində performans testləri aparılıb. Əsas mənbə: M. A. Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition).

Tapşırıq 1 – Aptekdə növbə (Queue): src/pharmacy_queue.py

Tapşırıq 2 – Avtobus dayanacağında prioritet növbə: src/bus_stop.py

Testlər src/benchmark.py ilə aparılıb, nəticələr test_results/ qovluğundadır. Hazır konteynerlər (collections.deque, queue.*) istifadə edilməyib; bütün strukturlar öz Node sinifləri ilə, göstərici (reference) əlaqələri ilə sıfırdan implementasiya edilib.

Tapşırıq 1. Aptekdə Növbə (Queue)

A. Araşdırma

Niyə aptek növbəsi üçün Queue strukturu uyğundur?

Aptekdə ilk gələn müştəri ilk xidmət almalıdır; Queue-nun FIFO prinsipi bunu birbaşa təmin edir və xidmət ardıcıllığının pozulması mümkün deyil.

FIFO prinsipi nə deməkdir?

First-In-First-Out: elementlər növbəyə rear (quyruq) ucundan daxil olur, front (baş) ucundan çıxarılır; çıxma ardıcıllığı daxil olma ardıcıllığı ilə eynidir.

Növbə niyə massiv yox, linkli siyahı ilə qurulub?

Massivdə başdan çıxarma (pop(0)) bütün elementləri sürüşdürür (O(n)); linkli növbədə front və rear göstəriciləri sayəsində həm əlavə, həm çıxarma O(1)-dir.

Stack istifadə olunsaydı nə baş verərdi?

Sonuncu gələn müştəri ilk xidmət alardı (LIFO), bu isə real növbə məntiqinə zidd olardı.

B. Proqramlaşdırma

CustomerNode – Name (ad), Medicine (dərman), next (göstərici).

PharmacyQueue – front, rear, size. Funksiyalar: addCustomer(), serveCustomer(), displayQueue(), countCustomers().

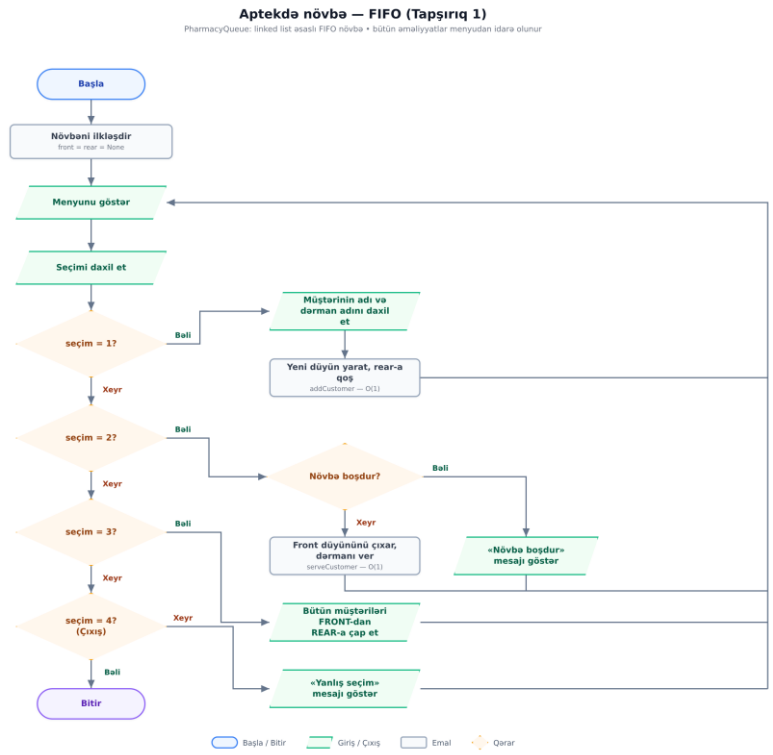
Bütün əsas əməliyyatlar O(1)-dir. Menyu: Add Customer, Serve Next Customer, Display Queue, Count Customers, Exit.

Şəkil: screenshots/task1_test.png

Psevdokod (əsas funksiyalar):

```
FUNKSIYA addCustomer(ad, dərman)                    // O(1)
    düyün ← yeni CustomerNode(ad, dərman)
    ƏGƏR rear = NULL ONDA front ← düyün             // növbə boş idi
    YOXSA rear.next ← düyün                          // sona əlavə
    rear ← düyün
    size ← size + 1
SON

FUNKSIYA serveCustomer()                             // O(1)
    ƏGƏR front = NULL ONDA QAYTAR NULL               // növbə boşdur
    düyün ← front
    front ← düyün.next                               // başdan çıxar
    ƏGƏR front = NULL ONDA rear ← NULL               // son düyün çıxarıldı
    size ← size – 1
    QAYTAR düyün
SON
```



Şəkil 1. Tapşırıq 1 – aptek növbəsinin blok-sxemi

C. Təcrübə və Eksperiment

100, 1000 və 10000 müştəridən ibarət növbələrdə Add (enqueue) və Serve (dequeue) əməliyyatlarının N əməliyyatlı ümumi vaxtı ölçülüb (5 təkrar, median, time.perf_counter). Nəticələr Cədvəl 1 və Şəkil 3-dədir.

Tapşırıq 2. Avtobus Dayanacağında Prioritet Növbə

A. Araşdırma

Prioritet növbə nədir?

Elementlərin xidmət sırası gəliş vaxtına yox, prioritetinə görə müəyyən olunan növbədir; bu tapşırıqda yaşlılar (65 yaşdan yuxarı) prioritetli sayılır.

Prioritet necə modelləşdirilib?

İki müstəqil FIFO növbəsi istifadə olunub – prioritetli (65+) və normal. addPassenger() yaşı yoxlayıb sərnişini uyğun növbəyə yerləşdirir.

Niyə tək sıralı siyahı yox, iki ayrı növbə seçilib?

Sıralı siyahıda əlavə üçün yer axtarışı O(n) olardı; iki növbədə isə hər iki əməliyyat O(1)-dir və hər növbə daxilində FIFO qorunur.

Avtobusa minmə (board) əməliyyatı necə işləyir?

Əvvəlcə prioritetli növbə yoxlanılır; o boşdırsa normal növbədən götürülür, hər ikisi boşdursa xəbərdarlıq çap olunur. Bütün hallar O(1)-dir.

B. Proqramlaşdırma

PassengerNode – Name (ad), Age (yaş), next. PassengerQueue – front, rear (köməkçi FIFO növbəsi).

BusStop – priorityQueue, normalQueue (PRIORITY_AGE = 65). Funksiyalar: addPassenger(), boardPassenger(), displayPassengers(), countPassengers().

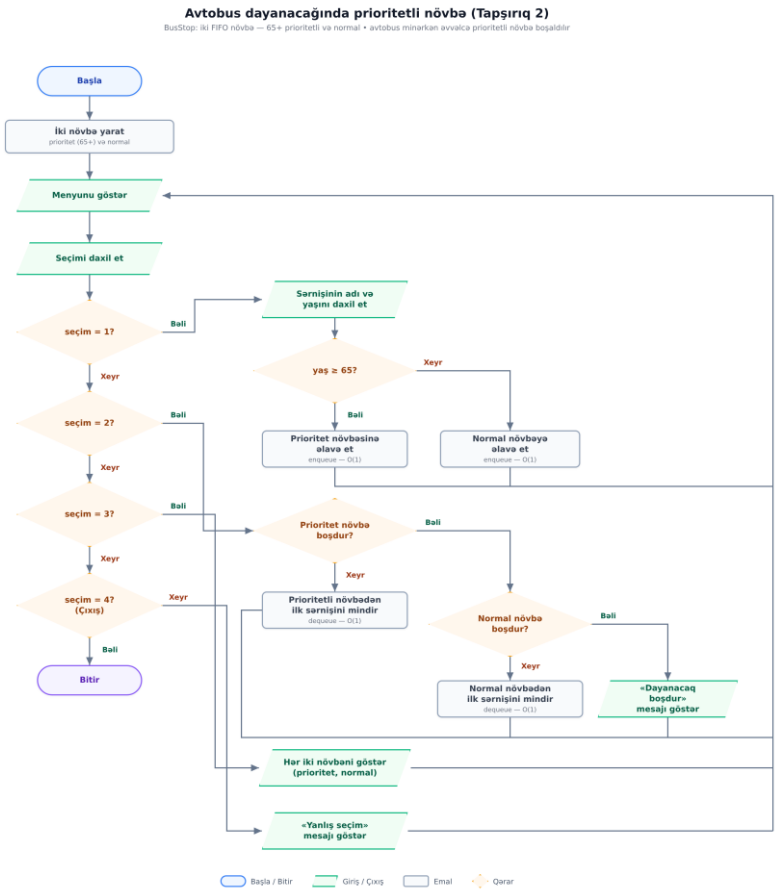
Add və Board O(1)-dir. Menyu: Add Passenger, Board Next Passenger, Display Queues, Count Passengers, Exit.

Şəkil: screenshots/task2_test.png

Psevdokod (əsas funksiyalar):

```
FUNKSIYA addPassenger(ad, yaş)                    // O(1)
    ƏGƏR yaş ≥ 65 ONDA
        priorityQueue.enqueue(ad, yaş)            // prioritetli növbə
    YOXSA
        normalQueue.enqueue(ad, yaş)              // normal növbə
    SON ƏGƏR
SON

FUNKSIYA boardPassenger()                         // O(1)
    ƏGƏR priorityQueue boş DEYİL ONDA
        QAYTAR priorityQueue.dequeue()            // prioritetli minir
    SON ƏGƏR
    ƏGƏR normalQueue boş DEYİL ONDA
        QAYTAR normalQueue.dequeue()              // normal növbədən minir
    SON ƏGƏR
    QAYTAR NULL                                   // dayanacaq boşdur
SON
```



Şəkil 2. Tapşırıq 2 – prioritetli dayanacağın blok-sxemi

C. Təcrübə və Eksperiment

100, 1000 və 10000 sərnişində Add və Board əməliyyatlarının ümumi vaxtı ölçülüb (5 təkrar, median, time.perf_counter). Nəticələr Cədvəl 2 və Şəkil 4-dədir.

Test nəticələri

Hər proqramın menyusundakı bütün əməliyyatlar yoxlanılıb və gözlənilən nəticələri verib. Prioritet məntiqi xüsusi yoxlanılıb: yaşlı (65+) sərnişinlər həmişə normal növbədəki sərnişinlərdən əvvəl avtobusa minib. Performans testlərinin nəticələri Cədvəllər və Qrafiklər bölmələrində verilib.

Şəkil: screenshots/task1_test.png

Şəkil: screenshots/task2_test.png

Şəkil: screenshots/benchmark_output.png

Sistem məlumatları:

Parametr	Dəyər
Computer Model	Lenovo LOQ 15
CPU	Intel Core i7-14700HX (28 məntiqi nüvə)
RAM	16 GB
GPU	NVIDIA GeForce RTX 4060
Operating System	Windows 11 (10.0.26100)
Compiler / Interpreter	Python (CPython)
Compiler Version	3.14.3
IDE	Visual Studio Code
Screen Resolution	1920x1080
Storage Type (SSD/HDD)	SSD
Power Mode	Performance (şarj cihazına qoşulu)
Other Factors	Python tək nüvədən istifadə edir; ölçmələr 5 təkrarın medianıdır

Cədvəllər

Cədvəl 1. Aptek növbəsi (Queue)

Add və Serve – N əməliyyatın ümumi vaxtı (5 təkrarın medianı).

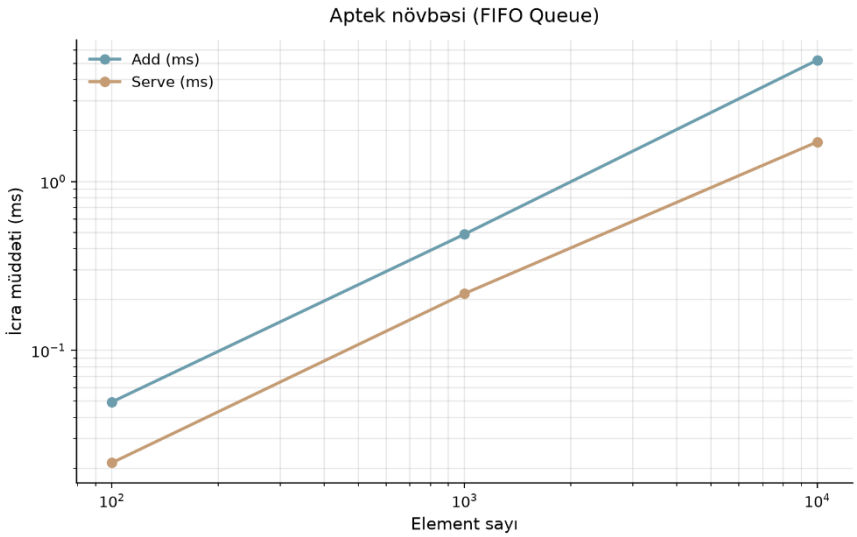
Element sayı	Add (ms)	Serve (ms)
100	0.0406	0.0189
1000	0.4527	0.1973
10000	5.0515	2.0983

Cədvəl 2. Avtobus dayanacağı (Prioritet növbə)

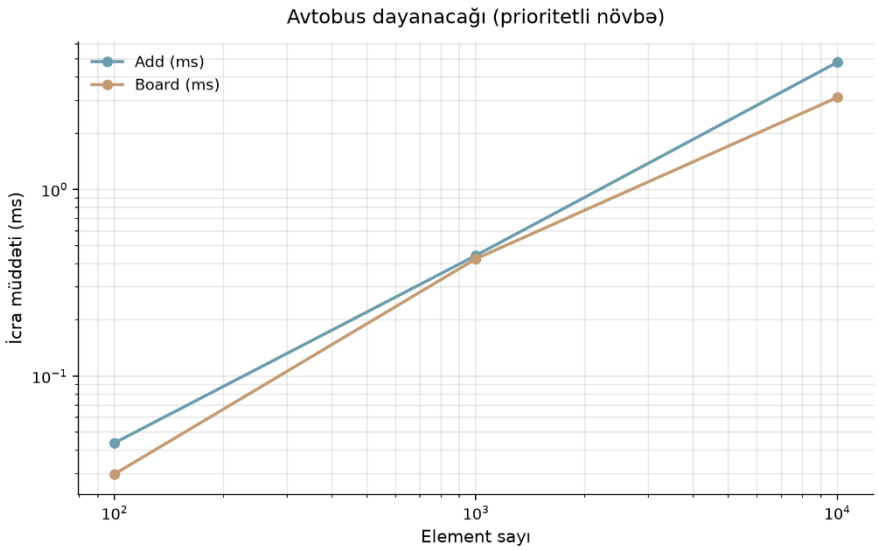
Element sayı	Add (ms)	Board (ms)
100	0.0452	0.0362
1000	0.4922	0.3716
10000	5.4199	5.175

Qrafiklər

X oxu element (əməliyyat) sayı, Y oxu icra müddətidir (ms). Hər iki ox loqarifmikdir.



Şəkil 3. Aptek növbəsi: Add və Serve



Şəkil 4. Avtobus dayanacağı: Add və Board

Analiz

Tapşırıq 1

Serve həmişə Add-dan sürətli olub: N=10000 üçün Add 5.05 ms, Serve 2.10 ms (təxminən 2.4 dəfə). Serve yeni düyün yaratmır, yalnız front göstəricisini irəli aparır.

Bəli. N 10 dəfə artdıqda Add 11.2 və 11.2 dəfə, Serve 10.4 və 10.6 dəfə artıb – ümumi vaxt N ilə xətti artır, yəni bir əməliyyat O(1)-dir.

Massivlə (Python list) qurulsaydı, pop(0) bütün elementləri sürüşdürdüyü üçün Serve O(n), N əməliyyatın ümumi vaxtı isə $O(n^2)$ olardı; N=10000-də fərq çox böyük olardı.

Boş növbə halları ayrıca yoxlanılıb: boş növbədə Serve xəta yox, “Növbə boşdur” mesajı verir; son düyün çıxarılanda rear göstəricisi NULL-a qaytarılır.

Tapşırıq 2

Board, Add-a yaxın sürətlə işləyir: N=10000 üçün Add 5.42 ms, Board 5.18 ms. Board əlavə yoxlama (prioritet kaskadı) aparsa da, hər iki əməliyyat yalnız göstəricilərlə işləyir.

Bəli. N 10 dəfə artdıqda Add 10.9 və 11.0 dəfə, Board 10.3 və 13.9 dəfə artıb – xətti artım O(1) əməliyyatları təsdiqləyir.

Prioritet iki müstəqil FIFO növbəsi ilə modelləşdirilib: addPassenger() yaşı yoxlayır (≥ 65), boardPassenger() prioritet kaskadı ilə çıxarır. Hər ikisi O(1)-dir.

Tək sıralı siyahı istifadə olunsaydı, əlavə üçün yer axtarışı O(n) olardı. Çatışmazlıq: yeni prioritet səviyyəsi lazım olanda ayrıca növbə yaratmaq tələb olunur.

Sistem parametrlərinin təsiri

CPU tezliyi və keş vaxtları birbaşa təyin edir; linkli siyahı düyünləri yaddaşda səpələndiyi üçün keşə həssaslıq var.

Arxa plan prosesləri və enerji qənaəti rejimi vaxtları artıra bilər; buna görə median götürülüb.

Python interpretator olduğu üçün C++ ilə müqayisədə hər əməliyyat daha bahalıdır.

Kompüterdə 28 məntiqi nüvə olsa da, Python kodu (GIL) bir nüvədə işləyir, ona görə nüvə sayı nəticələri sürətləndirmir.

RAM, GPU, Storage və ekran ayırdetməsinin təsiri bu testlərdə minimaldır, çünki fayl və qrafik əməliyyatı ölçülmür.

Nəticə

Hər iki tapşırıqda bütün əsas əməliyyatlar (Add, Serve, Board) O(1) mürəkkəbliyi ilə icra olundu: N 10 dəfə artdıqda ümumi vaxt da təxminən 10 dəfə artdı. Aptek ssenarisi təmiz FIFO prinsipini, avtobus dayanacağı isə iki FIFO növbəsinin birləşməsi ilə qurulmuş prioritet məntiqini göstərdi; sosial prioritet (65+) əlavə yaddaş və vaxt xərci olmadan modelləşdirildi.

İstinadlar

Weiss, M. A. Data Structures and Algorithm Analysis in C++, 4th Edition. Pearson, 2014.

Goodrich, M. T., Tamassia, R., Goldwasser, M. H. Data Structures and Algorithms in Python. Wiley, 2013.

Python documentation, time.perf_counter: https://docs.python.org/3/library/time.html

AI Usage Declaration

Tool: Claude AI

Purpose: Kodların yazılması və araşdırmalardan sonra məlumatların toplu halda yazılması

Used Sections: Hesabatın mətn hissəsi (Giriş, Araşdırma, Analiz, Nəticə). Kodlar tələbə tərəfindən sıfırdan yazılıb.

TƏLƏBƏ ÜÇÜN MƏLUMAT

Əziz tələbələr, DSA laboratoriyasına xoş gəlmisiniz. Bu kurs ərzində siz ən azı səkkiz (8) laboratoriya tapşırığını yerinə yetirməlisiniz. Bu praktiki məşğələlər proqramlaşdırma bacarıqlarınızı möhkəmləndirmək, nəzəri biliklərinizi gücləndirmək və problem həll etmə qabiliyyətinizi inkişaf etdirmək məqsədi daşıyır. Hər bir laboratoriya işi tamamlandıqdan sonra sizdən hesabat (lab report) hazırlanması və təhvil verilməsi tələb olunacaq. Hazırladığınız hesabatlar müəlliminiz tərəfindən yoxlanılacaq və qiymətləndiriləcək.

1. Laboratoriyanın məqsədləri

DSA laboratoriyası aşağıdakı məqsədlərlə hazırlanmışdır:

Müxtəlif verilənlərin strukturlarını və alqoritmləri tətbiq etmək üzrə praktiki təcrübə qazandırmaq.

Strukturlaşdırılmış tapşırıqlar vasitəsilə analitik və səhv axtarma (debugging) bacarıqlarını inkişaf etdirmək.

Aydın və peşəkar texniki hesabatların hazırlanması vərdişlərini formalaşdırmaq.

2. Laboratoriyaya hazırlıq

Laboratoriya məşğələsinə gəlməzdən əvvəl:

Laboratoriya təlimatını (manual) və məqsədləri diqqətlə oxuyun.

Verilmiş proqramlaşdırma tapşırıqlarını müstəqil şəkildə həll etməyə çalışın.

Alqoritm, psevdokod və blok-sxem eskizlərinizi (qaralamalarınızı) əvvəlcədən hazırlayın.

Bu hazırlıq laboratoriya vaxtının səmərəli istifadəsini təmin edir və məşğələ zamanı proqramın yoxlanması, səhvlərin aradan qaldırılması və təkmilləşdirilməsinə fokuslanmağa kömək edir.

3. Laboratoriya hesabatının (Lab Report) təqdim edilməsi

Hər bir laboratoriya işi üçün siz aşağıdakı struktur əsasında ətraflı hesabat hazırlayıb təqdim etməlisiniz. Əgər təqdim etmə gecikərsə, əvvəlcədən müəllimin icazəsi alınmadığı halda qiymət azaldıla bilər.

Laboratoriya hesabatının strukturu:

Üz Qapağı (Cover Page)

Aşağıdakı məlumatlar göstərilməlidir:

Fənnin adı, laboratoriya nömrəsi, laboratoriyanın mövzusu

Tələbənin adı, tələbə nömrəsi (ID), qrup nömrəsi

Müəllimin adı, təqdim etmə tarixi

Başlıq (Title)

Laboratoriya işinin əsas məqsədini dəqiq ifadə edin. (Məsələn: “Massivlərdən istifadə etməklə stek əməliyyatlarının reallaşdırılması”)

Məqsədlər (Objectives)

Laboratoriyanın məqsədlərini və öyrənmə nəticələrini qısa şəkildə təsvir edin.

Problemin təhlili (Problem Analysis)

Problemi aydın şəkildə müəyyənləşdirin və aşağıdakılara diqqət edin:

Giriş və çıxış verilənlərinin təsviri

Məhdudiyyətlər və gözlənilən nəticələr

İcra üçün seçilmiş verilənlərin strukturu və ya alqoritm

Alqoritm / Psevdokod (Algorithm / Pseudocode)

Alqoritmi addım-addım təqdim edin. Oxunaqlılığı təmin etmək üçün standart psevdokod formatından istifadə edin.

Blok-sxem (Flowchart)

Alqoritmin qrafik təsvirini göstərin. Standart simvollardan istifadə edin və məntiqi ardıcıllığı qoruyun (Başla → Giriş → Emal → Çıxış → Bitir).

Proqramlaşdırma (Coding)

Kodu təmiz, oxunaqlı və sənədləşdirilmiş şəkildə yazın. Düzgün indentasiya (boşluq) və adlandırma qaydalarına əməl edin. Əsas funksiyaları və məntiqi hissələri izah edən qısa şərhlər (comment) əlavə edin.

Nəticələr və Test (Output and Testing)

Proqramın işləmə nəticələrini müxtəlif test nümunələri ilə göstərin. Əgər test zamanı xəta və ya problem yaranıbsa, onu necə aradan qaldırdığınızı qeyd edin.

Müzakirə və Nəticə (Discussion & Conclusion)

Laboratoriya zamanı nə öyrəndiyinizi ümumiləşdirin. Qarşılaşdığınız çətinlikləri, apardığınız təkmilləşdirmələri və gələcəkdə inkişaf imkanlarını müzakirə edin.

Müəllim rəyi bölməsi (Instructor Feedback Section)

Bu hissə müəllimin şərhləri və qiymətləndirmə meyarları üçün ayrılmışdır:

Qiymətləndirmə meyarı	Faiz (%)
Problemin təhlili	10%
Alqoritm və blok-sxem	20%
Kodlaşdırma və funksionallıq	30%
Hesabatın keyfiyyəti	20%
Müzakirə və nəticə	20%

4. Peşəkar yanaşma (Professional Practice)

Akademik dürüstlüyü qoruyun — plagiat (başqasının işindən icazəsiz istifadə) cəza ilə nəticələnəcəkdir.

Hesabatlarınızı düzgün formatda və qrammatik cəhətdən düzgün şəkildə tərtib edin.

Laboratoriya rəhbərliyində təqdim olunan “Tam laboratoriya hesabatı necə yazılır” adlı nümunə sənədi formatlaşdırma üçün bələdçi kimi istifadə edin.

Məsələ 1: Asan — Aptekdə növbə

Aptekdə insanlar dərman almaq üçün növbəyə düzülürlər. Hər müştəri növbəyə ad və tələb olunan dərman adı ilə daxil olur. Müştəri dərmanını aldıqdan sonra növbədən çıxarılır.

Növbəyə yeni müştəri əlavə edən funksiyanı yaz.

Növbədəki ilk müştərini dərmanını verdikdən sonra çıxaran funksiyanı yaz.

Növbədəki bütün müştəriləri ekrana çıxaran funksiyanı yaz.

Bu, klassik FIFO (First-In-First-Out) prinsipidir. İlk gələn ilk xidmət alır. Növbə strukturunda yeni müştəri sona əlavə olunur, xidmət alan isə başdan çıxarılır.

Məsələ 2: Orta — Avtobus dayanacağında növbə və prioritet

Avtobus dayanacağında insanlar növbəyə düzülürlər. Lakin yaşlı insanlar (65 yaşdan yuxarı) prioritetlə avtobusa minirlər. Digərləri isə normal növbə ilə minir.

Növbəyə yeni sərnişin əlavə edən funksiyanı yaz (ad və yaş daxil edilir).

Avtobusa minmək üçün növbədəki ilk prioritetli sərnişini çıxaran funksiyanı yaz. Əgər prioritetli sərnişin yoxdursa, normal növbədən çıxar.

Növbədəki sərnişinlərin siyahısını ekrana çıxaran funksiyanı yaz.

Burada iki növbə işlədilir — biri yaşlılar üçün, digəri digərləri üçün. Avtobus gələndə əvvəlcə yaşlılar növbəsindən sərnişin götürülür. Bu, real həyatda sosial prioritetin proqramlaşdırma ilə necə modelləşdirildiyini göstərir.